

# 大一学生学习部署手册

## 极度稀疏矩阵的低秩补全 与潜在特征发现

从零开始理解 *Soft-Impute* 与 *ALS*

配套代码 + 实验步骤 + 老师问答模拟

适用对象：线性代数/高等代数课程大一学生

前置知识：矩阵运算、特征值与特征向量（了解即可）

2026 年 5 月 17 日

# 目录

|          |                                |           |
|----------|--------------------------------|-----------|
| <b>1</b> | <b>写在前面：你需要知道什么</b>            | <b>2</b>  |
| <b>2</b> | <b>问题背景：为什么需要矩阵补全？</b>         | <b>3</b>  |
| 2.1      | 一个直观的例子 . . . . .              | 3         |
| 2.2      | 关键观察：用户不是随便打分的 . . . . .       | 3         |
| <b>3</b> | <b>SVD：把矩阵拆开来看</b>             | <b>4</b>  |
| 3.1      | 什么是 SVD? . . . . .             | 4         |
| 3.2      | 为什么 SVD 能发现潜在特征? . . . . .     | 4         |
| 3.3      | Python 中的 SVD . . . . .        | 5         |
| <b>4</b> | <b>Soft-Impute：用 SVD 填补缺失值</b> | <b>6</b>  |
| 4.1      | 问题难点：有缺失值怎么做 SVD? . . . . .    | 6         |
| 4.2      | 软阈值：比直接截断更聪明 . . . . .         | 6         |
| 4.3      | 核心迭代公式 . . . . .               | 7         |
| 4.4      | Soft-Impute 代码逐行解读 . . . . .   | 8         |
| <b>5</b> | <b>ALS：另一种思路</b>               | <b>10</b> |
| 5.1      | 换个角度：直接优化两个矩阵 . . . . .        | 10        |
| 5.2      | 为什么叫”交替”最小二乘? . . . . .        | 10        |
| 5.3      | 最小二乘的闭式解 . . . . .             | 10        |
| <b>6</b> | <b>手把手：跑通实验</b>                | <b>12</b> |
| 6.1      | 文件结构 . . . . .                 | 12        |
| 6.2      | 运行步骤 . . . . .                 | 12        |
| 6.3      | main.py 的工作流程 . . . . .        | 12        |
| <b>7</b> | <b>老师问答模拟（必看!）</b>             | <b>14</b> |
| <b>8</b> | <b>常见错误与排查</b>                 | <b>17</b> |
| <b>9</b> | <b>拓展阅读与进阶方向</b>               | <b>18</b> |

# 1 写在前面：你需要知道什么

## 目标

本手册的目标是让一位刚学完基础线性代数的大一学生，能够：

1. 理解”矩阵补全”问题的实际意义；
2. 知道 SVD 是什么、为什么它能提取”潜在特征”；
3. 看懂 Soft-Impute 和 ALS 的代码每一行在做什么；
4. 自己动手跑通实验，并能回答老师的课堂提问。

你需要的前置知识（如果不会，先补一下）：

- 矩阵乘法、转置、Frobenius 范数  $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$
- 特征值、特征向量的基本定义（知道  $Av = \lambda v$  即可）
- Python 基础（numpy 数组操作）

你不需要立刻懂的东西（可以先放一放）：

- 核范数的严格凸分析证明
- ALS 的收敛性理论保证（如 Incoherence 条件）
- 随机矩阵论中的 RIP 条件

## 2 问题背景：为什么需要矩阵补全？

### 2.1 一个直观的例子

想象你是一家视频网站的推荐算法工程师。网站有 1000 个用户、2000 部电影。你把所有可能的评分放进一张大表格里：

|      | 电影 A | 电影 B | 电影 C | 电影 D |
|------|------|------|------|------|
| 用户 1 | 5    | ?    | 3    | ?    |
| 用户 2 | ?    | 4    | ?    | 2    |
| 用户 3 | 1    | ?    | 5    | ?    |

问号表示“这个用户还没看过/没评过”。问题是：如何猜测这些问号里的数字？

### 2.2 关键观察：用户不是随便打分的

用户 1 给电影 A 打了 5 分、电影 C 打了 3 分。用户 3 给电影 A 打了 1 分、电影 C 打了 5 分。这背后可能有某种规律：

- 用户 1 可能喜欢“动作大片”，电影 A 是动作片所以给高分，电影 C 是文艺片所以给低分
- 用户 3 可能喜欢“文艺片”，所以电影 A（动作片）低分，电影 C（文艺片）高分

如果我们能找出这些**隐藏的类型**（动作、文艺、喜剧……），就可以预测缺失评分。这就是**低秩假设**：评分矩阵  $M$  可以近似写成两个“瘦矩阵”的乘积：

$$M \approx UV^T \quad (1)$$

其中  $U$  的每一行是一个用户的“类型偏好向量”， $V$  的每一行是一部电影的“类型含量向量”。

#### 核心假设

真实评分矩阵是**低秩**的。也就是说，虽然矩阵看起来很大（ $943 \times 1682$ ），但真正决定评分的可能只有 5-20 个“潜在维度”。

### 3 SVD：把矩阵拆开来看

#### 3.1 什么是 SVD?

SVD（奇异值分解，Singular Value Decomposition）是线性代数中最强大的工具之一。它说：任何矩阵  $M$  都可以拆成三个矩阵的乘积：

$$M = U\Sigma V^T \quad (2)$$

形象地理解（假设  $M$  是  $m \times n$ ）：

- $U$  是  $m \times m$  的正交矩阵，每一列是一个”左奇异向量”（可以理解为用户的”纯类型”）
- $\Sigma$  是  $m \times n$  的对角矩阵，对角线上的  $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$  叫**奇异值**（表示每个类型的”重要性”）
- $V^T$  是  $n \times n$  的正交矩阵，每一行是一个”右奇异向量”（可以理解为电影的”纯类型”）

#### 3.2 为什么 SVD 能发现潜在特征?

如果把式 (2) 展开，它其实是：

$$M = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_r u_r v_r^T \quad (3)$$

每一项  $\sigma_k u_k v_k^T$  都是一个秩为 1 的矩阵，代表一种”潜在模式”。例如：

- 第 1 项可能对应”动作片偏好 vs. 动作片含量”
- 第 2 项可能对应”文艺片偏好 vs. 文艺片含量”
- .....

因为奇异值  $\sigma_k$  通常**快速衰减**（第一项很大，第二项小很多，第三项更小……），所以只用前几个项就能很好地近似原矩阵。这就是**低秩近似**：

$$M_k = \sum_{i=1}^k \sigma_i u_i v_i^T \approx M \quad (4)$$

### 一句话记忆

SVD 把矩阵拆成一堆”类型模式”的叠加，每个模式有一个重要性权重（奇异值）。丢掉不重要的模式，就得到了低秩近似。

## 3.3 Python 中的 SVD

Listing 1: Numpy 中的 SVD

```
1 import numpy as np
2
3 # 对一个矩阵做 SVD
4 M = np.array([[1, 2], [3, 4], [5, 6]]) # 3x2 矩阵
5 U, s, Vt = np.linalg.svd(M, full_matrices=False)
6
7 print(U.shape) # (3, 2)
8 print(s.shape) # (2,) 奇异值数组
9 print(Vt.shape) # (2, 2)
10
11 # 重构低秩近似（保留前 k 个）
12 k = 1
13 M_approx = (U[:, :k] * s[:k]) @ Vt[:k, :]
```

## 4 Soft-Impute: 用 SVD 填补缺失值

### 4.1 问题难点：有缺失值怎么做 SVD？

如果矩阵是完整的，直接做 SVD 然后截断就行。但实际问题中很多条目是”?”（缺失值）。

Soft-Impute 的思路非常直观：

1. 先用某种猜测值（比如全局平均分）填满所有”?”
2. 对填满后的矩阵做 SVD，保留前几个奇异值（扔掉小的）
3. 用新的低秩矩阵替换原来的猜测，回到第 1 步
4. 重复直到收敛

### 4.2 软阈值：比直接截断更聪明

直接截断 SVD 是”硬截断”：把小的奇异值直接变成 0。Soft-Impute 用的是”软阈值”：

$$(d - \lambda)_+ = \max(d - \lambda, 0) \quad (5)$$

这意味着：

- 如果奇异值  $d > \lambda$ ，就把它缩小  $\lambda$ （而不是直接删掉）
- 如果奇异值  $d \leq \lambda$ ，就把它变成 0

#### 软阈值 vs. 硬截断

硬截断：  $d = 5, 3, 1, 0.5$ ，保留前 2 个  $\rightarrow 5, 3, 0, 0$

软阈值 ( $\lambda = 2$ )：  $5 \rightarrow 3, 3 \rightarrow 1, 1 \rightarrow 0, 0.5 \rightarrow 0$

软阈值的好处是：

- 自动选择有效秩（小奇异值被压到 0）
- 大奇异值被”收缩”，起到正则化作用，防止过拟合
- 整个问题是凸优化，保证收敛到全局最优

### 4.3 核心迭代公式

$$Z^{\text{new}} = S_{\lambda} \left( \underbrace{P_{\Omega}(M)}_{\text{真实值}} + \underbrace{P_{\Omega}^{\perp}(Z^{\text{old}})}_{\text{缺失位置用旧估计}} \right) \quad (6)$$

用大白话说：

1. 构造一个新矩阵：有数据的地方用真实值，没数据的地方用上一轮猜的值
2. 对这个新矩阵做 SVD + 软阈值
3. 得到新的猜测，进入下一轮



## 4.4 Soft-Impute 代码逐行解读

以下为核心代码（来自 `src/soft_impute.py`），我们逐段讲解：

Listing 2: Soft-Impute 核心迭代（简化版）

```
1 def soft_impute(M, Omega, lambda_param=15.0, max_iter=60):
2     # M: 评分矩阵, 缺失值是 np.nan
3     # Omega: bool 矩阵, True 表示"这个位置有数据"
4
5     # 初始化: 用全局平均分填充所有缺失位置
6     observed = M[Omega]          # 取出所有真实评分
7     mean_val = np.mean(observed) # 计算全局平均分
8     X = np.full(M.shape, mean_val)
9     X[Omega] = observed          # 有数据的位置用真实值
10
11     for it in range(max_iter):
12         # 第1步: 构造  $Z = P_{\Omega}(M) + P_{\Omega}^{\perp}(X)$ 
13         Z = X.copy()
14         Z[Omega] = observed      # 有数据的位置用真实值
15
16         # 第2步: 对 Z 做 SVD
17         U, s, Vt = np.linalg.svd(Z, full_matrices=False)
18
19         # 第3步: 软阈值
20         s_thresh = np.maximum(s - lambda_param, 0.0)
21
22         # 第4步: 用阈值后的奇异值重构低秩矩阵
23         r = np.sum(s_thresh > 0) # 有效秩 (非零奇异值个数)
24         X_new = (U[:, :r] * s_thresh[:r]) @ Vt[:, :r]
25
26         # 检查收敛
27         change = np.linalg.norm(X_new - X, 'fro')
28         if change < 1e-4:
29             break
30         X = X_new
31
32     return X
```

### 调试技巧

运行时可以打印每轮迭代的 `change` 和有效秩 `r`。如果 `change` 越来越小但 `r` 稳定，说明算法在收敛；如果 `r` 一直跳变，可能需要调大或调小  $\lambda$ 。

## 5 ALS: 另一种思路

### 5.1 换个角度：直接优化两个矩阵

Soft-Impute 是”先补全再分解”。ALS 走另一条路：直接假设  $M \approx UV^T$ ，然后找最好的  $U$  和  $V$ 。

优化目标：

$$\min_{U,V} \sum_{(i,j) \in \Omega} (M_{ij} - u_i^T v_j)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2) \quad (7)$$

大白话：让  $U$  的第  $i$  行（用户  $i$  的偏好）乘以  $V$  的第  $j$  行（电影  $j$  的含量），尽可能接近真实评分  $M_{ij}$ 。

### 5.2 什么叫”交替”最小二乘？

式 (7) 同时对  $U$  和  $V$  优化是非凸的（双线性），很难直接解。但如果：

- 固定  $V$ ，只优化  $U \rightarrow$  变成很多个独立的最小二乘问题（每个用户一行）
- 固定  $U$ ，只优化  $V \rightarrow$  同样变成很多个独立的最小二乘问题（每部电影一行）

所以算法”交替”进行：

1. 随机初始化  $U$  和  $V$
2. 固定  $V$ ，用最小二乘更新  $U$  的每一行
3. 固定  $U$ ，用最小二乘更新  $V$  的每一行
4. 重复 2-3 直到收敛

### 5.3 最小二乘的闭式解

对用户  $i$ ，已知  $V$ ，求最佳  $u_i$ ：

$$u_i = (V_{\Omega_i}^T V_{\Omega_i} + \lambda I)^{-1} V_{\Omega_i}^T M_{i,\Omega_i} \quad (8)$$

其中  $\Omega_i$  是用户  $i$  评过分的电影集合。这个公式看起来吓人，其实就是在解一个线性方程组  $Ax = b$ ，其中：

- $A = V_{\Omega_i}^T V_{\Omega_i} + \lambda I$  ( $k \times k$  小矩阵)
- $b = V_{\Omega_i}^T M_{i,\Omega_i}$  ( $k$  维向量)

Listing 3: ALS 核心更新（用户侧）

```
1 for i in range(m): # 对每个用户
2     items = user_items[i] # 该用户评过分的电影索引列表
3     if len(items) == 0:
4         continue
5     V_i = V[items, :] # 取出这些电影的 V 行
6     M_i = M[i, items] # 取出这些评分
7
8     # 解最小二乘:  $(V_i^T V_i + \lambda I) u_i = V_i^T M_i$ 
9     A = V_i.T @ V_i + reg_lambda * np.eye(rank)
10    b = V_i.T @ M_i
11    U[i, :] = np.linalg.solve(A, b)
```

### ALS 的陷阱

- ALS 是非凸的，结果依赖初始化，可能陷入局部最优
- rank  $k$  需要手动设置，太大容易过拟合
- 每次迭代要解很多个小线性方程组，虽然每个很快，但总体不一定比 Soft-Impute 快

## 6 手把手：跑通实验

### 6.1 文件结构

Listing 4: 项目目录结构

```
1 d:\高代/  
2 |-- ml-100k/ml-100k/      # 数据集  
3 |-- src/  
4 |   |-- data_loader.py  
5 |   |-- soft_impute.py  
6 |   |-- als.py  
7 |   |-- metrics.py  
8 |   |-- experiments.py  
9 |-- results/              # 输出图表  
10 |-- main.py               # 主脚本
```

### 6.2 运行步骤

1. 确保已安装 Python 和 numpy, scipy, matplotlib, pandas
2. 打开终端，进入项目根目录：cd d:\高代
3. 运行：python main.py
4. 等待约 1-2 分钟，查看 results/ 目录下生成的图表

### 6.3 main.py 的工作流程

Listing 5: main.py 核心流程（简化）

```
1 # 1. 加载数据  
2 ratings_df, n_users, n_items = load_movielens_100k("ml-100k/ml-100k/u  
   .data")  
3 R_full = build_rating_matrix(ratings_df, n_users, n_items)  
4  
5 # 2. 划分训练/测试  
6 R_train, R_test, _, _ = mask_matrix(R_full, observed_indices, 0.8)  
7  
8 # 3. 运行 Soft-Impute  
9 R_pred_si, _ = soft_impute_simple(R_train, lambda_param=15.0)
```

```
10
11 # 4. 运行 ALS
12 R_pred_als, U, V, _ = als_simple(R_train, rank=10)
13
14 # 5. 评估
15 print(rmse(R_pred_si, R_test))
16 print(rmse(R_pred_als, R_test))
17
18 # 6. 做各种对比实验并画图
```

## 7 老师问答模拟（必看！）

以下问题基于真实的高等代数/线性代数课程课堂提问风格整理。建议通读并理解每个答案的核心要点。

### 为什么评分矩阵可以假设是低秩的？

**答：**用户偏好并非随机独立。喜欢科幻的人往往也喜欢动作片；喜欢文艺片的人评分风格相似。这意味着用户和电影都可以被归入少数几个“类型”（潜在特征），因此评分矩阵  $M$  可以近似为  $UV^T$ （ $U$  是用户特征， $V$  是电影特征），其中  $U, V$  的列数  $k$  远小于矩阵维度。这就是低秩假设。

**一句话版本：**因为少数几个隐藏类型就能解释大部分评分行为。

### SVD 和低秩近似有什么关系？

**答：**SVD 将矩阵  $M$  分解为  $U\Sigma V^T$ 。如果我们只保留前  $k$  个最大的奇异值，就得到  $M_k = U_k \Sigma_k V_k^T$ 。Eckart-Young-Mirsky 定理证明：在所有秩不超过  $k$  的矩阵中， $M_k$  是  $M$  的最佳 Frobenius 范数近似。因此 SVD 截断天然就是低秩近似。

### 核范数 $\|Z\|_*$ 是什么？为什么它能代替秩？

**答：**核范数是矩阵所有奇异值之和： $\|Z\|_* = \sum_i \sigma_i(Z)$ 。它与秩的关系类似于  $\ell_1$  范数和  $\ell_0$  范数的关系：

- $\text{rank}(Z)$  = 非零奇异值的个数（ $\ell_0$  型）
- $\|Z\|_*$  = 奇异值之和（ $\ell_1$  型）

$\ell_1$  是  $\ell_0$  的凸松弛，核范数是秩的凸松弛。优化核范数问题是凸的，可以用凸优化工具高效求解，而直接优化秩是 NP-hard 的。

### Soft-Impute 中的软阈值 $S_\lambda$ 在做什么？

**答：**对 SVD 得到的奇异值  $d_i$ ，软阈值操作是  $(d_i - \lambda)_+ = \max(d_i - \lambda, 0)$ 。它有两个效果：

1. 小于  $\lambda$  的奇异值被压到 0  $\rightarrow$  自动降秩
2. 大于  $\lambda$  的奇异值被缩小  $\lambda \rightarrow$  正则化/收缩

这样既保留了主要结构，又抑制了噪声，还能自动选择合适的秩。

**Soft-Impute 和 ALS 各有什么优缺点？**

答：

**Soft-Impute:**

- 优点：凸优化，保证全局收敛；自动选秩；对超参数不敏感
- 缺点：每步要做 SVD，计算量较大；对于超大规模矩阵需要特殊加速

**ALS:**

- 优点：每步只是解很多个小最小二乘，容易并行；工业界成熟（Netflix Prize 用过）
- 缺点：非凸，可能局部最优；需要手动选 rank；rank 过大容易过拟合

**为什么我们的实验中 ALS 的 RMSE 比 Soft-Impute 差？**

答：原因可能是多方面的：

1. ALS 是非凸优化，随机初始化可能不够好，陷入局部最优
2. ALS 的 rank  $k$  需要精细调参，我们用的是固定值  $k = 10$ ，未必最优
3. MovieLens-100k 数据量较小（仅 10 万条），ALS 的正则化效果不如核范数方法的“软收缩”精细
4. Soft-Impute 的核范数正则化在所有秩上同时收缩，具有更好的模型选择能力

**实验中的潜在特征（latent features）有实际意义吗？**

答：有。我们对 ALS 分解出的  $V$  矩阵进行了分析，发现：

- 维度 1 正向最强的电影是《FairyTale》《Les Misérables》等文艺/家庭片，负向是《Mr. Wrong》《Kingpin》等低俗喜剧。这对应“严肃情感深度 vs. 轻松娱乐”。
- 维度 2 正向是《The Stupids》《Cabin Boy》等另类片，负向是《Transformers》《Michael Collins》等主流片。这对应“独立/小众 vs. 主流商业”。

这说明低秩分解确实提取到了可解释的用户偏好模式。



### 奇异值谱分析说明了什么？

**答：**对评分矩阵（缺失值用均值填充后）做 SVD，发现奇异值快速衰减：第一个奇异值很大（约 4000），前 10 个就降到约 30。这说明：

- 矩阵的主要能量集中在少数几个奇异值上
- 去掉次要奇异值不会造成太大信息损失
- 低秩假设在实际数据中是成立的

这是矩阵补全问题“有解”的经验基础。

### 如果缺失率是 99%，这些方法还能工作吗？

**答：**理论上，如果矩阵足够低秩且观测位置随机均匀采样，即使缺失率极高（如 99%），在满足一定样本复杂度条件下仍然可以恢复（Candès & Recht, 2009）。但实践中：

- 误差会显著增大（我们的实验也显示 RMSE 随缺失率上升）
- 对算法的正则化强度要求更高
- 真实数据往往不是完全随机缺失（如用户倾向于评价自己喜欢的电影），这会带来系统性偏差

### 这个课题和 Netflix 推荐系统有什么关系？

**答：**Netflix Prize 是矩阵补全问题的经典应用。Netflix 有 48 万用户、1.8 万电影，观测评分仅占约 1.2%。获胜方案（BellKor）大量使用了矩阵分解技术（与 ALS 类似），结合其他方法（时间效应、邻居模型等）。我们的课题本质上是 Netflix 问题的“微缩版”，使用更小的数据集（MovieLens-100k）来验证核心算法的有效性。

## 8 常见错误与排查

### 错误 1: ModuleNotFoundError

现象: ModuleNotFoundError: No module named 'numpy'

解决: 安装依赖包

```
pip install numpy scipy matplotlib pandas
```

### 错误 2: 找不到数据集

现象: FileNotFoundError: ml-100k/ml-100k/u.data

解决: 确保在 d:\高代目录下运行 python main.py, 而不是在 src 子目录下运行。

### 错误 3: ALS 结果每次运行都不一样

现象: ALS 的 RMSE 每次运行有微小差异。

解决: 代码中已经设置了随机种子 `np.random.RandomState(42)`, 如果修改了代码中的种子, 结果会不同。这是正常的, 因为 ALS 依赖随机初始化。

### 错误 4: Soft-Impute 运行很慢

现象: Soft-Impute 迭代一次需要很长时间。

解决:

- 检查是否在做“完整 SVD” ( $943 \times 1682$  的完整 SVD 其实很快)
- 如果矩阵更大, 可以设置 `rank` 参数用稀疏 SVD 加速
- 减少 `max_iter` 或增大 `tol`

## 9 拓展阅读与进阶方向

如果你想在这个课题上做得更深入，可以考虑：

1. **加权正则化**：对不同用户或不同电影设置不同的正则化强度
2. **时间效应**：Netflix 数据中用户偏好会随时间变化，可以加入时间因子
3. **冷启动问题**：新用户/新电影没有评分，如何利用 side information（如用户年龄、电影类型）
4. **非负矩阵分解（NMF）**：要求  $U, V$  的元素都非负，使得潜在特征更易解释
5. **深度学习推荐模型**：如 AutoEncoder、Neural Collaborative Filtering

---

祝你学习顺利，课程项目拿高分！